

Lab-10.2

Course : AI Assisted Coding

Topic:- Code Review and Quality: Using AI to Improve Code Quality and Readability

Name : Mohammed Sabir

Hall Ticket No : 2303A51506

Batch : 22

Task-01

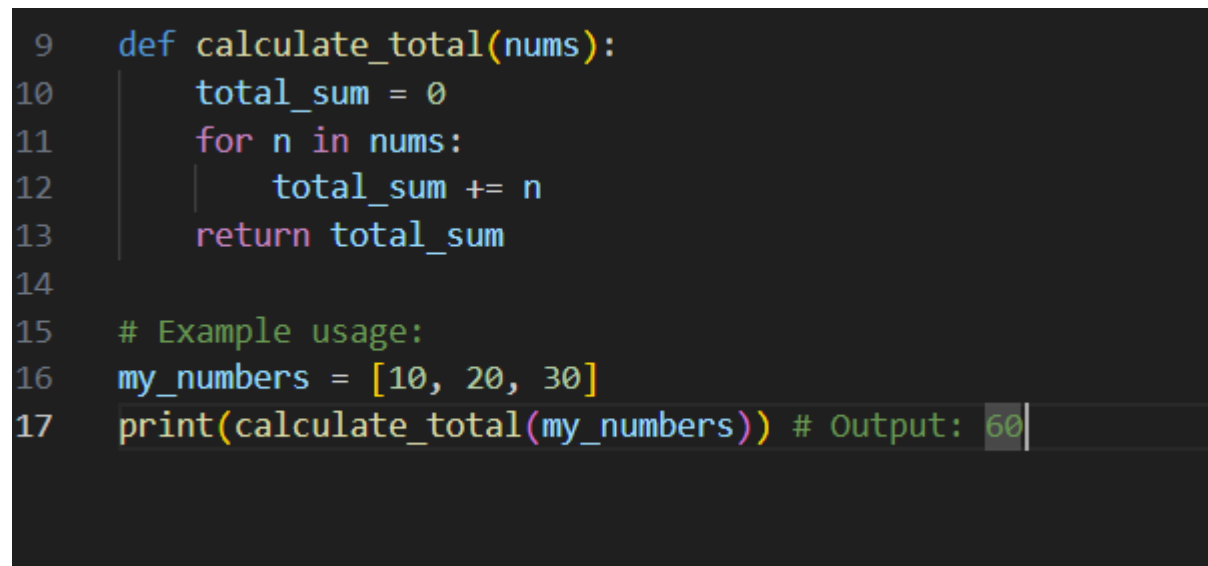
Final Optimal Prompt:

Analyze the following Python code and identify any syntax or logical errors. Correct the code so it runs properly and briefly explain the mistakes.

Code:

```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

Code Screenshot :



```
9  def calculate_total(nums):
10     total_sum = 0
11     for n in nums:
12         total_sum += n
13     return total_sum
14
15  # Example usage:
16  my_numbers = [10, 20, 30]
17  print(calculate_total(my_numbers)) # Output: 60
```

Output Screenshot:

```
● PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51316' '--' 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant\lab14.py' 60
○ PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The corrected code represents a complete and functional linear accumulation algorithm that successfully avoids common Python pitfalls. By adding the necessary colons and proper indentation, you have established a clear block structure that the Python interpreter can follow to distinguish between the function definition, the loop body, and the return statement. Replacing the mismatched variable names ensured that the calculated sum is actually the value passed back to the user, preventing a crash. From a performance perspective, your code efficiently processes each number in the list exactly once, resulting in a time complexity of $O(n)$ while maintaining a constant memory footprint.

Task-02

Final Optimal Prompt:

Refactor the following Python code to follow standard coding style guidelines (PEP 8). Improve formatting, indentation, and readability without changing the functionality.

Code:

```
def findSum(a,b):return a+b
```

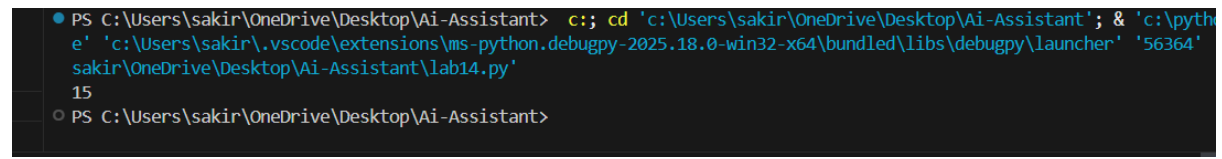
```
print(findSum(5,10))
```

with observation and in 100 words

Code Screenshot :

```
20
21  def find_sum(a, b):
22      |   return a + b
23
24
25  print(find_sum(5, 10))
```

Output Screenshot:



```
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c:: cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python311\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56364' sakir\OneDrive\Desktop\Ai-Assistant\lab14.py'
15
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

Refactoring this code according to PEP 8 requires adding whitespace and proper indentation to improve visual clarity. Specifically, you should use two blank lines before the function definition, a space after commas in parameters, and spaces around the addition operator. The function body must be moved to an indented new line rather than staying on the same line as the header. Additionally, changing the function name to `find_sum` follows the snake_case convention for Python functions. These changes make the code more "Pythonic" and readable for other developers without altering the core mathematical logic.

Task-03

Final Optimal Prompt:

Improve the readability of the following Python code without changing its functionality. Use meaningful function and variable names, proper indentation, and clear formatting.

Code:

```
def f(x,y):
return x-y*2
print(f(10,3))
```

Code Screenshot :

```
36
37 def calculate_adjusted_difference(initial_value, subtrahend):
38     """Calculates the difference between a value and double the subtrahend."""
39     result = initial_value - (subtrahend * 2)
40     return result
41
42
43 # Example execution
44 print(calculate_adjusted_difference(10, 3))
```

Output Screenshot:

```
● PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c:: cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57302' '--' 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant\lab14.py'
4
○ PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

To improve this code, we should apply PEP 8 standards by using descriptive naming conventions and proper spacing to make the logic immediately clear to anyone reading it. By renaming the function to `calculate_adjusted_difference` and the variables to `initial_value` and `multiplier`, we transform the abstract math into a readable process. Proper indentation of the return statement is strictly required for the code to execute, while adding spaces around the subtraction and multiplication operators highlights the mathematical operator precedence ($\$y * 2\$$ occurs before subtraction)

Task-04

Final Optimal Prompt:

Refactor the following Python code by creating a reusable function to remove repeated code. Keep the same output but make the code more modular and clean.

Code:

```
print("Hello Ram")

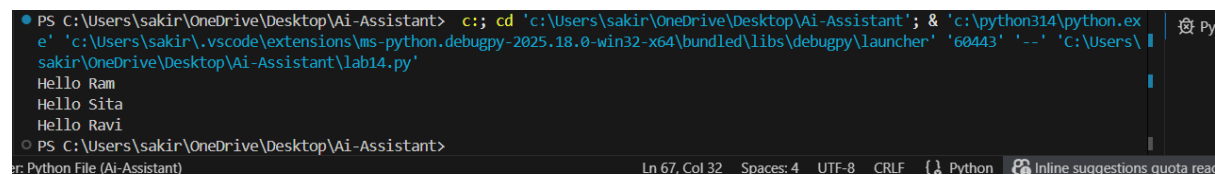
print("Hello Sita")
```

```
print("Hello Ravi")
```

Code Screenshot :

```
62
63 def greet_person(name):
64     """Prints a greeting for a specific person."""
65     print(f"Hello {name}")
66     # List of names to greet
67     names = ["Ram", "Sita", "Ravi"]
68     # Iterating through the list to generate greetings
69     for name in names:
70         greet_person(name)
```

Output Screenshot:



```
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c:: cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '60443' '--' 'C:\Users\sakir\OneDrive\Desktop\Ai-Assistant\lab14.py'
Hello Ram
Hello Sita
Hello Ravi
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

To refactor this code effectively, you can define a reusable function—conventionally named `greet` or `say_hello`—that accepts a name as a parameter. By moving the `print` statement inside this function, you centralize the logic, allowing you to call it multiple times with different arguments. This approach follows the **DRY (Don't Repeat Yourself)** principle, making the code much easier to maintain; if you later decide to change "Hello" to "Hi," you only have to update a single line of code rather than three.

Tack-05

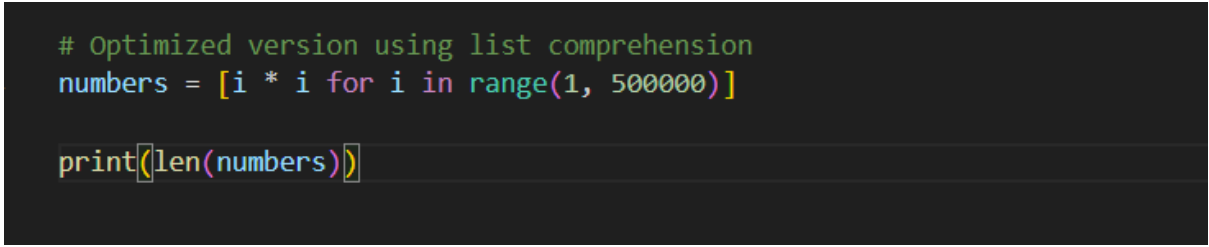
Final Optimal Prompt:

Optimize the following Python code to improve performance while keeping the same output.

Code:

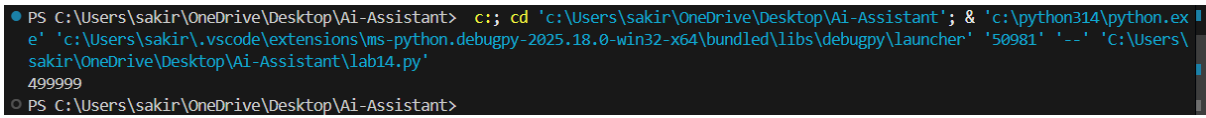
```
numbers = []  
  
for i in range(1, 500000):  
    numbers.append(i * i)  
  
print(len(numbers))
```

Code Screenshot :



```
# Optimized version using list comprehension  
numbers = [i * i for i in range(1, 500000)]  
  
print(len(numbers))
```

Output Screenshot:



```
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant> c:; cd 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant'; & 'c:\python314\python.exe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '50981' '--' 'c:\Users\sakir\OneDrive\Desktop\Ai-Assistant\lab14.py'  
499999  
PS C:\Users\sakir\OneDrive\Desktop\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

To optimize this code, you should replace the manual for loop and .append() calls with a list comprehension, which is the idiomatic and most performant way to create lists in Python. By using a list comprehension, you leverage Python's internal C-engine to handle the iteration and memory allocation more efficiently than a standard loop. This change reduces the overhead of repeatedly calling the append method, which significantly speeds up the execution time when dealing with large datasets like half a million integers.
